GOMPERTZ MODELING OF TUMOR GROWTH WITH A NONLINEAR CARRYING CAPACTIY

MEDEU AMANGELDI
ZACH MOBILLE

Abstract. INSERT ABSTRACT HERE

1. Introduction

Cancer is a leading cause of death in the United States. It is second only to heart disease. The National Cancer Institute reports that nearly 600,000 Americans died of cancer in 2017 [1] and that 16.4 million deaths are expected to be cancer-related in 2040. The disease is characterized by cells that divide uncontrollably, comprising tumors, that spread and debilitate healthy organ function.

The utility of mathematical models in understanding tumor growth and other cancer dynamics is difficult to overstate. The Gompertz equation is a simple model for population growth that has already been successfully applied to describing tumor growth data from lung and breast cancer tissues [2]. The carrying capacity in the Gompertz equation is typically taken to be a constant parameter. In certain cases, it is reasonable to instead assume that the carrying capacity is a function of the current tumor size. When a tumor is spreading, the possiblility of further growth tends to increase as a result of the cancer cells gaining more access to surrounding resources. In such a situation, the carrying capacity is an increasing function of tumor size [3]. Conversely, a tumor that is being treated by anti-cancer therapies has a carrying capacity that is either a constant or decreasing function of tumor size.

Here, we explore and test how the predictions of the Gompertz model for tumor growth are affected by various functional forms of the carrying capacity. The models are validated with various parameter estimation methods with respect to the tumor volume data of Vaghi et al. (2020) [2]. Focusing on the model with a carrying capacity that is an exponential function of tumor size, we find that distinct stability regimes exist for various parameter values. It is also shown through sensitivity analysis of the exponential carrying capacity model that the primary evolution equation shifts its sensitivity from the growth rate to the carrying capacity parameter as the tumor grows. This study provides a useful basis on which a more detailed investigation of tumor data may be conducted utilizing the Gompertz model with exponential carrying capacity.

2. Methods

The Gompertz equation is given by

$$\frac{dx}{dt} := f(x) = -rx\ln\left(\frac{x}{K}\right) \tag{1}$$

Date: March 24, 2026.

where $x \geqslant 0$ represents the tumor volume, $t \geqslant 0$ is time, $r > 0$ is a constant parameter describing the growth rate of the tumor, and $K > 0$ is the carrying capacity. K is given its name due to the following linear stability analysis of equation 1, where x_∞ are the fixed points.

$$\begin{aligned} 0 &= -rx_\infty \ln\left(\frac{x_\infty}{K}\right) \\ x_\infty &= 0, K \end{aligned}$$

Thus, $x_\infty = 0$ and $x_\infty = K$ are the fixed points of the Gompertz model with constant carrying capacity K. To obtain their stability, we compute:

$$f'(x) = -r\left[\ln\left(\frac{x}{K}\right) + 1\right]$$

Since $f'(x) \to \infty > 0$ as $x \to 0$, $x_\infty = 0$ is unstable. Likewise, we have that $f'(K) = -r < 0$, so $x_\infty = K$ is stable. Growth that is satisfied by equation 1 therefore always tends to $x = K$ as $t \to \infty$.
We have that $f'(x_{\max}) = 0$ when

$$\begin{aligned} 0 &= -r\left[\ln\left(\frac{x_{\max}}{K}\right) + 1\right] \\ \ln\left(\frac{K}{x_{\max}}\right) &= 1 \\ x_{\max} &= \frac{K}{e} \end{aligned}$$

Where $f(x) \leqslant f(x_{\max})$ for all x. We thus have that

$$\begin{aligned} f_{\max} := f(x_{\max}) &= -rx_{\max}\ln\left(\frac{x_{\max}}{K}\right) \\ &= -r\left(\frac{K}{e}\right)\ln\left(\frac{K/e}{K}\right) \\ &= \frac{rK}{e} \end{aligned}$$

Given these properties, the phase diagram of equation 1 is plotted in figure 1.

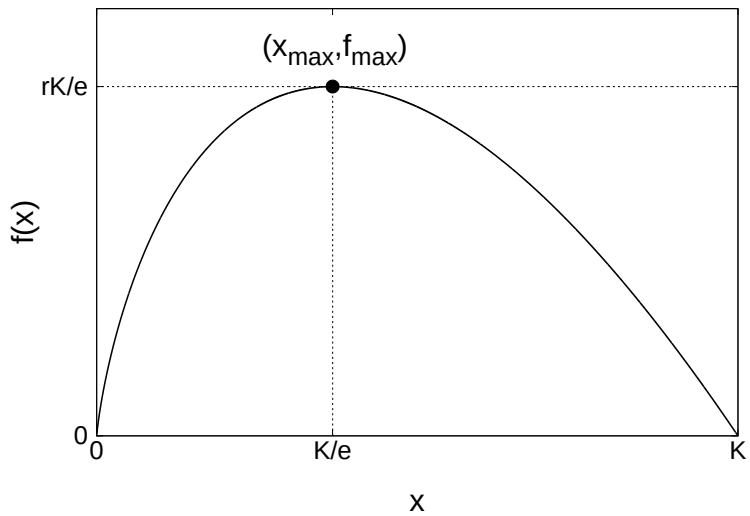

FIGURE 1. Example phase diagram of equation 1

2.1. **Analytical Solution for Gompertz Model with Constant Carrying Capacity.** Equation 1 can be solved directly. Separating and integrating, we have

$$\int \frac{dx}{x\ln(x/K)} = -r\int dt$$

Making the change of variable $u = \ln(x/K)$, this becomes

$$\begin{aligned}\int \frac{du}{u} &= -rt + C\\ \ln|u| &= -rt + C\\ |u| &= e^{-rt+C}\end{aligned}$$

For some integration constant C. Letting $A = e^C$ and back-transforming results in

$$\begin{aligned}|\ln(x/K)| &= Ae^{-rt}\\ \ln(x/K) &= \pm Ae^{-rt}\end{aligned}$$

Letting $A > 0$, the negative solution pertains to cases when $x(0) < K$ while the positive solution is appropriate for $x(0) > K$. Going further, we have

$$\begin{aligned}x/K &= e^{\pm Ae^{-rt}}\\ x(t) &= Ke^{\pm Ae^{-rt}}\end{aligned}$$

Imposing the initial condition $x(0) = x_0$ yields

$$x_0 = Ke^{\pm A}$$

Thus, the solution to equation 1 for constant K is

$$x(t) = K^{1-e^{-rt}}x_0^{e^{-rt}} \tag{2}$$

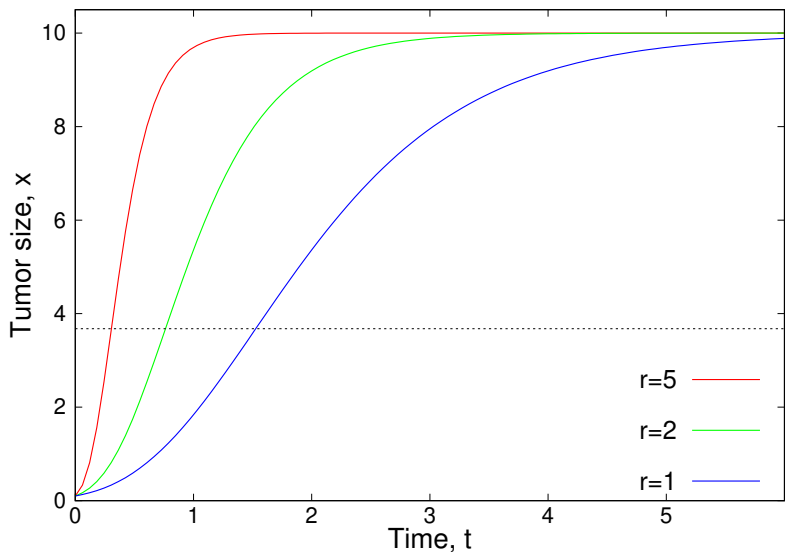

FIGURE 2. Equation 2 plotted for various values of r, with other parameters fixed at $K = 10$ and $x_0 = 0.1$. The dashed line shows the tumor size at which $\frac{d^2x}{dt^2} = 0$, which is $x = K/e \approx 10/2.71$

Equation 2 may be manipulated to find that a transformed version of the tumor size is a linear function in time. Taking the natural log of equation 2 yields

$$\begin{aligned} \ln x &= (1 - e^{-rt})\ln K + e^{-rt}\ln x_0 \\ &= \ln K - e^{-rt}\ln K + e^{-rt}\ln x_0 \\ \ln\left(\frac{x}{K}\right) &= e^{-rt}\ln\left(\frac{x_0}{K}\right) \\ \frac{\ln\left(\frac{x}{K}\right)}{\ln\left(\frac{x_0}{K}\right)} &= e^{-rt} \end{aligned}$$

Taking the natural log, again, yields

$$\ln\left[\frac{\ln\left(\frac{x}{K}\right)}{\ln\left(\frac{x_0}{K}\right)}\right] = -rt \tag{3}$$

Equation 3 is used in section 3.1 to fit a Gompertz model with constant carrying capacity to transformed tumor time series data.

2.2. **Relationships between the Gompertz and other Population Models.**

2.2.1. *Constant change model.* When we let the carrying capacity $K = xe^{c/x}$ for some non-zero constant c, the Gompertz equation becomes

$$\begin{aligned} \frac{dx}{dt} &= -rx\ln\left(\frac{x}{xe^{c/x}}\right) \\ &= rx\ln(e^{c/x}) \\ &= rc \end{aligned}$$

This is a constant change model with solution $x(t) = rct + x_0$ where $x(0) = x_0$. This results in linear growth when $c > 0$ and linear decay when $c < 0$.

2.2.2. *Exponential growth model.* Suppose instead that we let $K = cx$ for some positive constant c. The Gompertz model becomes

$$\begin{aligned}\frac{dx}{dt} &= -rx\ln\left(\frac{x}{cx}\right)\\ &= -rx\ln(1/c)\end{aligned}$$

This is an exponential model with solution $x(t) = x_0 e^{-r\ln(1/c)t}$ where $x(0) = x_0$. This solution predicts exponential growth when $c > 1$ and exponential decay when $c < 1$.

2.2.3. *Logistic growth model.* Suppose we make a change of variable $y + 1 := x/K$. Equation 1 becomes

$$\frac{dy}{dt} = -r(y+1)\ln(y+1)$$

With Taylor's expansion $\ln(y+1) = y - \frac{y^2}{2} + \frac{2y^3}{6} - \cdots$, we may write

$$\frac{dy}{dt} = -r(y+1)[y + O(y^2)]$$

For small transformed tumor volumes y (such that we may reasonably neglect terms with $O(y^2)$), this allows us to approximate the Gompertz model as:

$$\frac{dy}{dt} \approx -ry(y+1) \tag{4}$$

which is topologically equivalent to the logistic model with carrying capacity $K = 1$, given by:

$$\frac{dy}{dt} = ry(1-y) \tag{5}$$

Both the approximation given by equation 4 and the logistic equation in 5 display transcritical bifurcations at $r = 0$. The logistic model may therefore be viewed as an approximation to the Gompertz model when tumor size is small.

Alternatively, we may see the Gompertz model as related to the generalized logistic model:

$$\frac{dx}{dt} = \frac{rx}{\alpha}\left[1 - \left(\frac{x}{K}\right)^{\alpha}\right]$$

For some $\alpha \neq 0$, $r > 0$. Taking the limit and invoking L'Hôpital's rule, we have that

$$\begin{aligned}\lim_{\alpha\to 0}\frac{dx}{dt} &= \lim_{\alpha\to 0}\frac{rx}{\alpha}\left[1 - \left(\frac{x}{K}\right)^{\alpha}\right]\\ &= rx\lim_{\alpha\to 0}\frac{\left[1 - \left(\frac{x}{K}\right)^{\alpha}\right]}{\alpha}\\ &= rx\lim_{\alpha\to 0}\left[-\left(\frac{x}{K}\right)^{\alpha}\ln\left(\frac{x}{K}\right)\right]\\ &= -rx\ln\left(\frac{x}{K}\right)\end{aligned}$$

which is equivalent to the original Gompertz model stated in equation 1.

2.3. **Survey of Various Functional Forms for the Nonlinear Carrying Capacity**

We now consider a modification of equation 1, letting $K = K(x)$. We are interested in a system of the form

$$\frac{dx}{dt} := f(x) = -rx\ln\left[\frac{x}{K(x)}\right] \tag{6}$$

with various formulas for the carrying capacity as a function of tumor size $K(x)$.

2.3.1. *Exponential carrying capacity.* Letting $K(x) = e^{-bx}$ for constant b gives

$$f(x) = -rx\ln\left(xe^{bx}\right) \tag{7}$$

$$= -rx\left[\ln x + bx\right] \tag{8}$$

Which yields tumor growth $\dot{x} := \frac{dx}{dt} > 0$ when

$$\ln\left(\frac{1}{x}\right) > bx$$

and tumor decay $\dot{x} < 0$ when

$$\ln\left(\frac{1}{x}\right) < bx$$

The fixed points x_∞ are found by

$$0 = -rx_\infty\left[\ln x_\infty + bx_\infty\right]$$

One fixed point is $x_\infty = 0$. It is always unstable, since

$$\begin{aligned} f'(x) &= -r(\ln x + bx) - rx\left(\frac{1}{x} + b\right) \\ &= -r(\ln x + 2bx - 1) \\ \lim_{x\to 0} f'(x) &= -r\lim_{x\to 0}(\ln x + 2bx - 1) \\ &= \infty \end{aligned}$$

The other fixed points are given by the solutions x_∞ to

$$bx_\infty = \ln\left(\frac{1}{x_\infty}\right) \tag{9}$$

A graphical solution to equation 9 for x_∞ is shown in figure 3, for the case $b > 0$.

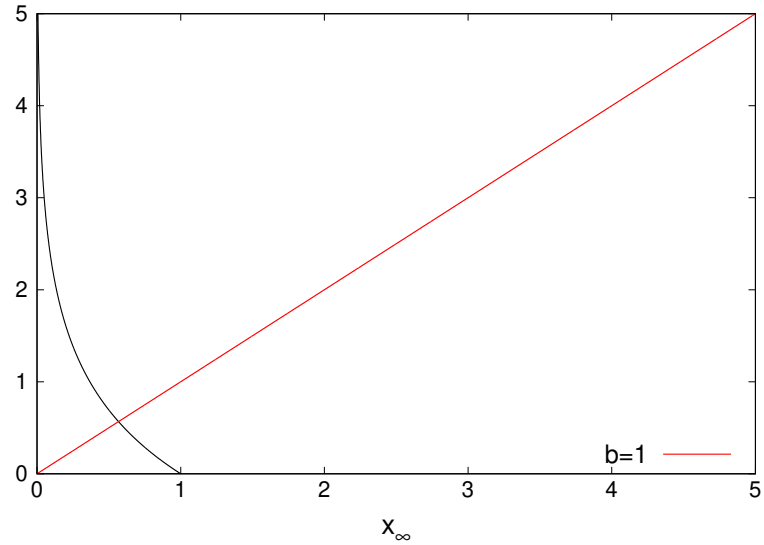

FIGURE 3. Example of a graphical solution to equation 9 when $b > 0$, given by the intersection of the curves. The black curve is $\ln(1/x_\infty)$. The red line is $y = bx$ with $b = 1$.

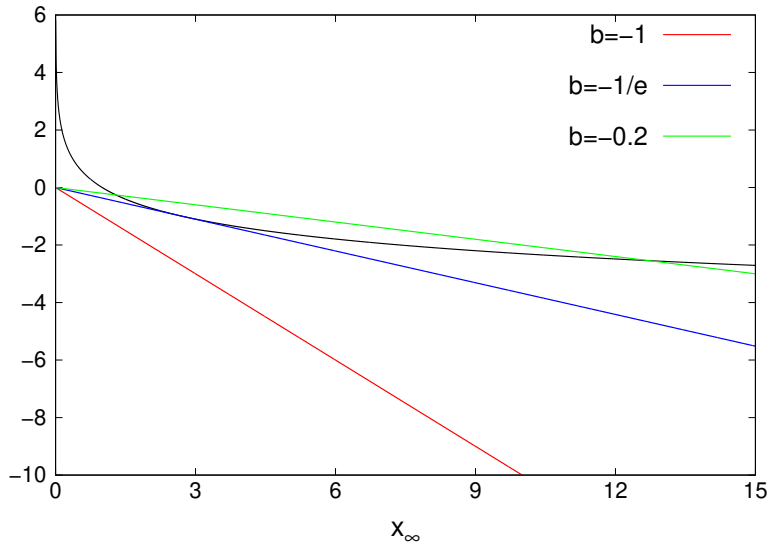

FIGURE 4. Possible graphical solutions to equation 9 for three different values of b, when $b < 0$. The black curve is $\ln(1/x_\infty)$. The non-black lines are $y = bx$ with the value of b given in the legend.

It can be seen that there is always a stable equilibrium when $b > 0$. So long as the slope of the line in figure 3 is positive, altering b will not change the stability of the equilibrium shown by the intersection point. It is always stable, because $\ln\left(\frac{1}{x}\right) > bx$ and hence $\dot{x} > 0$ for all $x < x_\infty$ while $\ln\left(\frac{1}{x}\right) < bx$ and hence $\dot{x} < 0$ for all $x > x_\infty$. Furthermore, the stable state

for $b > 0$ is always less than 1, since the red line may intersect the black logarithm in figure 3 at tumor sizes no greater than $x = 1$ when b is only allowed to run positive. On the other hand, if $b < 0$, then there are three possibilities. The first possibility is that the line $y = bx$ is too steep to intersect $y = -\ln x$ and there are no equilibria; i.e. the red line in figure 4. The second possibility is that the line $y = bx$ intersects $y = \ln\left(\frac{1}{x}\right)$ exactly once (blue line of figure 4), resulting in a half-stable equilibrium at $x = e$. The third possibility is that they intersect twice (green line of figure 4). The two-intersection case results in two equilibria.

The tangent intersection of $\ln(1/x)$ and bx occurs when equation 9 is satisfied, in addition to

$$\begin{aligned} \frac{d}{dx}\left[\ln(1/x)\right] &= \frac{d}{dx}(bx) \\ x(-x^{-2}) &= b \\ -1/x &= b \end{aligned}$$

Plugging this in to equation 9 yields

$$\begin{aligned} \left(-\frac{1}{x}\right)x &= \ln\left(\frac{1}{x}\right) \\ -1 &= \ln\left(\frac{1}{x}\right) \\ e &= x \end{aligned}$$

Thus, the line $y = \left(-\frac{1}{e}\right)x$ intersects $y = \ln(1/x)$ tangentially at $x = e$. When $b < -1/e$, there is no intersection. This case therefore implies that there are no positive equilibria when $b < -1/e$. There are two equilibria when $b > -1/e$. The smaller fixed point is stable, the larger one is unstable. This stability landscape differs from that of the Gompertz model with constant carrying capacity, for which an analytical solution was obtained in section 2.1. In that model, $x_\infty = 0$ is always unstable and $x_\infty = K$ is always stable. In the model analyzed here with an exponential carrying capacity, the value of b determines whether there are zero, one, or two stable states. In the case of two stable states ($-1/e < b < 0$), it is the smaller tumor size which is stable while the larger one is unstable.

Equation 7 has an implicit form solution, derived by the method of invariance under the one-parameter group and integrating factor (refer to Applications of Lie Groups to Differential Equations), which is

$$\log(brtx + rt\log(x) + 1) + \int \frac{1 - brtx - rt}{x(brtx + rt\log(x) + 1}dx = C \tag{10}$$

for some constant C. Moreover, by integration method from Lie group's theory, we have the following implicit solution as well

$$w = \int \frac{dy}{-rye^{w}\log(ye^{e^{w}(by+1)}) - 1} + C \tag{11}$$

where $w = \log(t)$, $y = \frac{x}{t}$, and C is a constant of integration.

Since we do not have an analytical solution to equation 7, we solve it numerically using the fourth-order Runge-Kutta method.

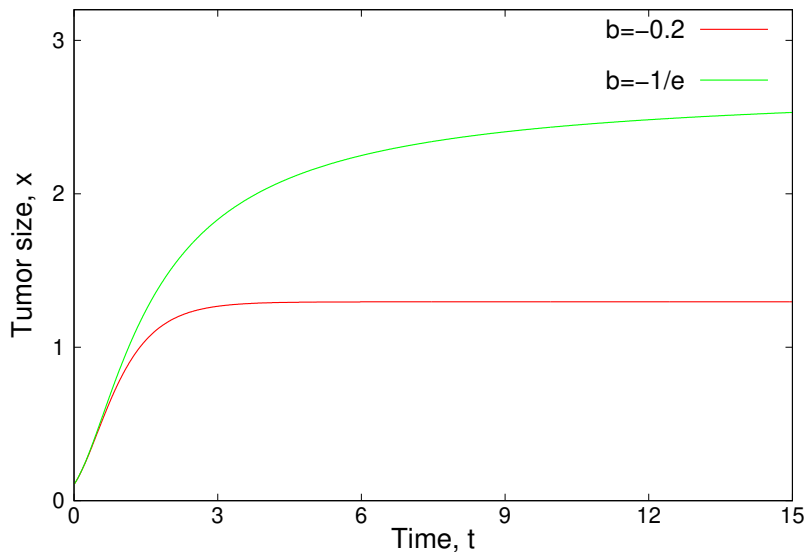

FIGURE 5. Numerical solutions to Gompertz model with exponential carrying capacity (equation 7); $r = 2$, $x_0 = 0.1$

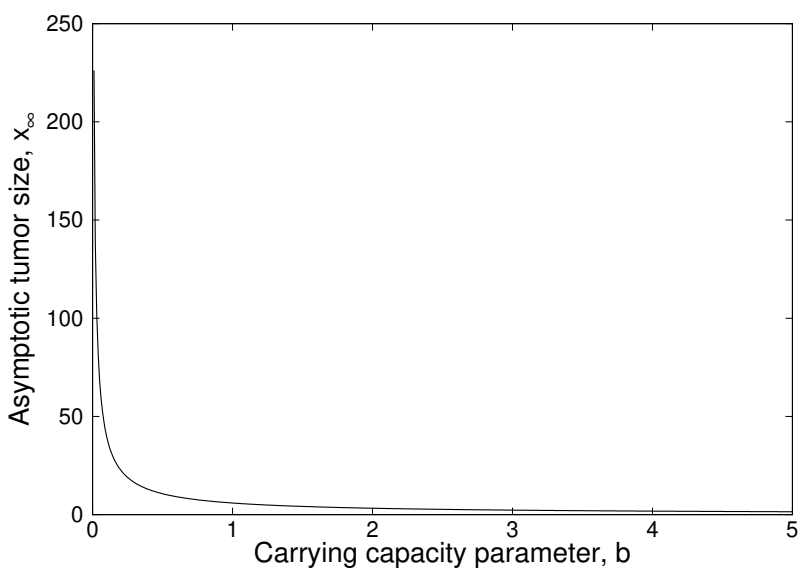

FIGURE 6. Bifurcation diagram for the model stated by equation 6 where $K(X) = 2171e^{-bx}$ with parameters $r = 0.082$ and $x_0 = 72.6$, motivated by the results from section 3.1.

2.3.2. *Polynomial carrying capacity.* Letting $K(x) = bx^2 + cx + d$ in equation 6 for constants b, c, d gives

$$\frac{dx}{dt} = -rx\ln\left(\frac{x}{bx^2 + cx + d}\right) \tag{12}$$

which still has fixed point $x_\infty = 0$. The non-zero fixed points are given by

$$0 = \ln x_\infty - \ln(bx_\infty^2 + cx_\infty + d) \tag{13}$$

$$ln(x_\infty) = ln(bx_\infty^2 + cx_\infty + d) \tag{14}$$

$$0 = bx_\infty^2 + (c-1)x_\infty + d \tag{15}$$

$$x_\infty = \frac{1 - c \pm \sqrt{(c-1)^2 - 4bd}}{2b} \tag{16}$$

where $b \neq 0$. For $b = -0.5$ and $c = 3$ and $d = 0$, the equilibrium points are shown in figure 7.

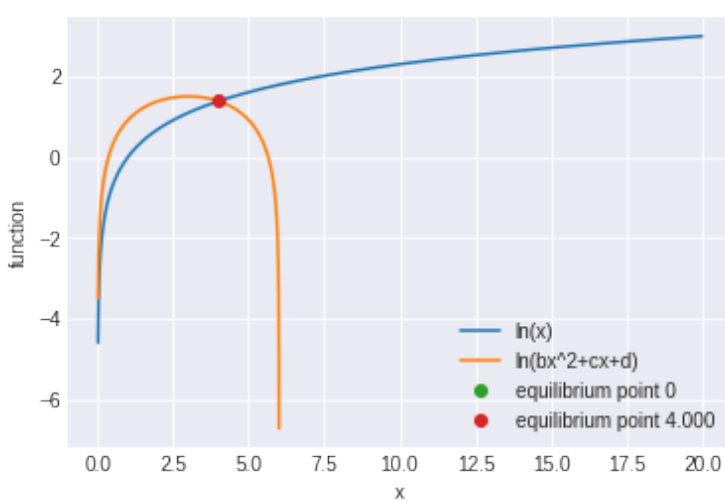

FIGURE 7. Intersection of two functions from equation 15 where $b = -0.5$, $c = 3$, and $d = 0$.

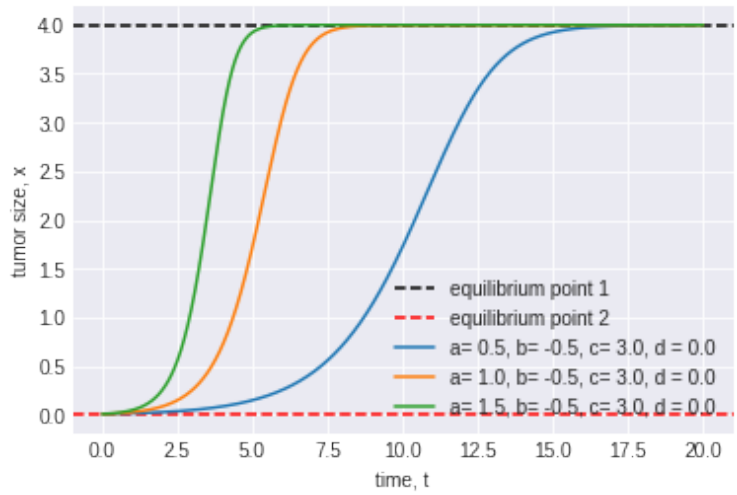

FIGURE 8. Numerical solutions to equation 12, with parameter values listed in the legend.

2.3.3. *Logarithmic carrying capacity.* Letting $K(x) = b\ln(cx)$ in equation 6 for constants b and c gives

$$\frac{dx}{dt} = -ax\ln\left(\frac{x}{bln(cx)}\right)$$

which yields

$$\frac{dx}{dt} = -ax\left[\ln x - \ln(bln(cx))\right] \tag{17}$$

The equilibria x_∞ is found by equating 17 to zero. Disregarding the equilibrium $x_\infty = 0$, we have that

$$ln(x_\infty) = ln\left[bln(cx_\infty)\right] \tag{18}$$

which is solved graphically in figure 9 below.

2.4. **Parameter Estimation**

2.4.1. *Cancer Data.* The time series data we work with are tumor volume measurements over 35 days from three different cancer-injected animals. The data displayed in figure 11 were retrieved using WebPlotDigitizer from the paper by Vaghi et al. (2020) [2].

2.4.2. *Least-Squares.* In this section, we construct the minimization problem of least-squares. In general, if $\{x_i\}_{i=1}^m$ is the set of m observations of tumor volume and $\{t_i\}_{i=1}^m$ is the corresponding set of m observations of days, then we are interested in minimizing the sum of squares:

$$S = \sum_{i=1}^{m}[x(\vec{\theta}, t_i) - x_i]^2 \tag{19}$$

where $\vec{\theta} = (\theta^1, \ldots, \theta^k)$ is the vector of k parameters and $x(\vec{\theta}, t_i)$ are the predicted tumor sizes at time t_i. We find $\vec{\theta}$ by solving the so-called "normal equations":

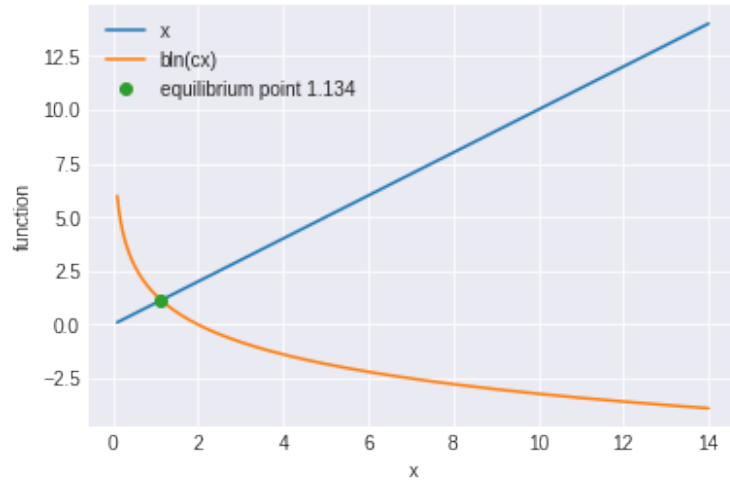

FIGURE 9. Graphical solution to equation 18

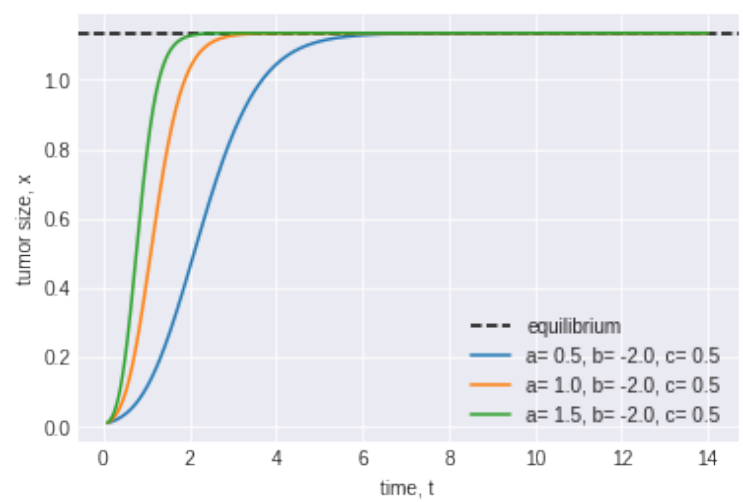

FIGURE 10. Numerical solution to equation 17, with parameter values shown in the legend.

$$\frac{\partial S}{\partial \theta^i} = 0 \qquad 1 \leqslant i \leqslant k \tag{20}$$

In the case of a constant carrying capacity K, an analytical solution for $x(\vec{\theta}, t_i)$ exists (eq. 2) and equation 20 can be solved by linear algebraic methods. Thus, estimates for $\vec{\theta}$ can be found directly in the case of constant K.

For the exponential carrying capacity model, we use Newton's method to solve equation 20 to find the estimated parameters. See section 3.3 for further details.

3. Results

3.1. Fitting a Constant K Model to Transformed Data

The Ordinary Least-Squares (OLS) method is used to fit data shown in figure 11, transformed according to equation 3, to the analytical solution with constant carrying capacity shown by equation 2.

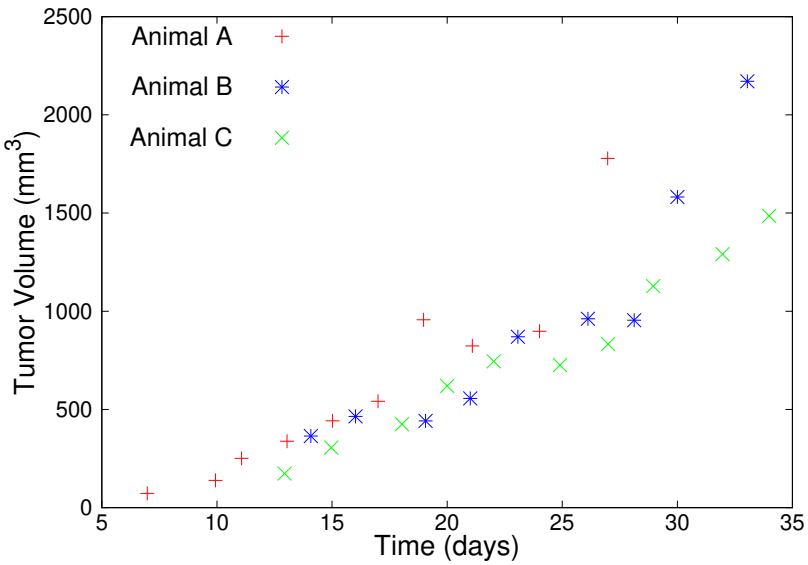

Figure 11. Tumor data for three different animals, adapted from [2]

Defining $\{x_i\}_{i=1}^{29}$ as the time-ordered set of the 29 observations plotted in figure 11, we set the initial condition x_0 on the left-hand side of equation 3 equal to the data point $x_1 = \min(x_i) \approx 72.6$ mm^3, occurring on the 7th day (taken as $t = 0$ in the model) for animal A. Additionally, we fix the carrying capacity K equal to $\max(x_i) \approx 2170.9$ mm^3 on the 33rd day from animal B. This leaves $29 - 2 = 27$ observations for fitting r on the right-hand side of equation 3. The best-fit line along with transformed data are shown in figure 12.

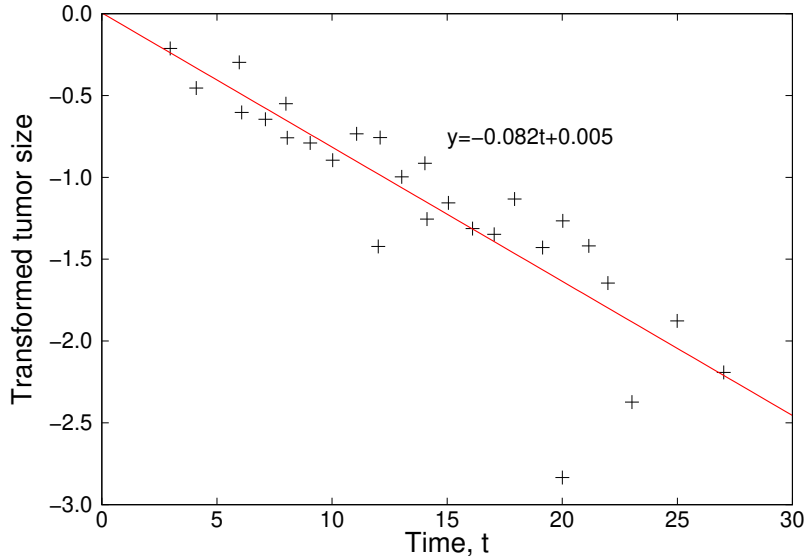

FIGURE 12. Best fit line to data shown in figure 11, transformed according to equation 3; $x_0 = 72.6$ and $K = 2170.9$. The red line shows the ordinary least-squares (OLS) fit, which yields a point estimate for the growth rate of $r = 0.082$ and 95% confidence interval (CI) of (0.044,0.121).

The Breusch-Pagan test was used on the residuals of the fit shown in figure 12, yielding a p-value of 0.1922. Thus, we conclude that there is no evidence of heteroscedasticity. The residual plot in figure 13 confirms this conclusion, visually.

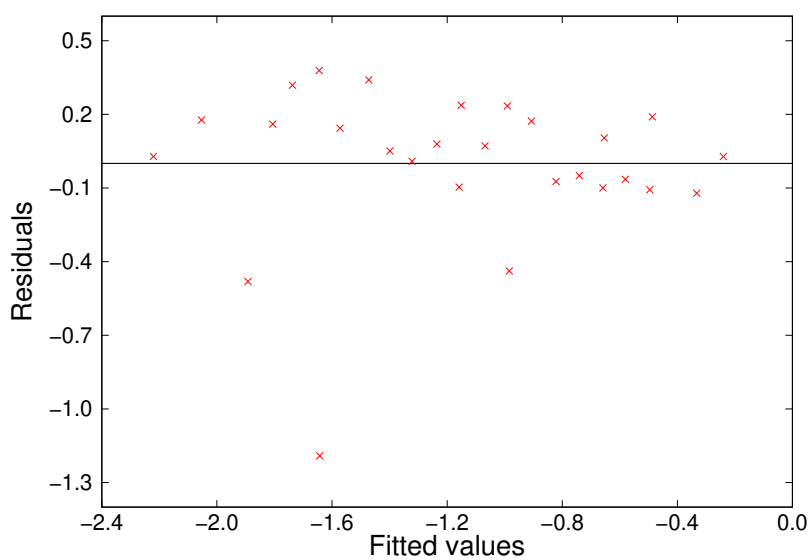

FIGURE 13. Residual plot for statistical fit shown in figure 12. The black horizontal line is $y = 0$.

3.2. **Local Sensitivity Analysis of Exponential Carrying Capacity Model.** Following the methodology outlined in [4], we define the sensitivity functions $\phi_r := \frac{\partial x}{\partial r}$ and $\phi_b := \frac{\partial x}{\partial b}$ for the Gompertz model with exponential carrying capacity, restated as

$$F(x; r, b) = -rx(\ln x + bx) \tag{21}$$

For a given sensitivity function $\phi_i = \phi_i(t)$ $(i = r, b)$, we have the evolution equation and accompanying initial condition

$$\begin{cases} \frac{d\phi_i}{dt} & = \frac{\partial F}{\partial x}\phi_i + \frac{\partial F}{\partial i} \\ \phi_i(0) & = 0 \end{cases}$$

Taking these with equation 21 yields the dynamical system

$$\frac{dx}{dt} = -rx(\ln x + bx) \tag{22}$$

$$\frac{d\phi_r}{dt} = -r\phi_r\left(\ln x + 2bx + 1\right) - x\ln x - bx^2 \tag{23}$$

$$\frac{d\phi_b}{dt} = -r\phi_b\left(\ln x + 2bx + 1\right) - rx^2 \tag{24}$$

with initial conditions $x(0) = x_0$, $\phi_r(0) = \phi_b(0) = 0$. Equations 22-24 are solved numerically in figure 14, using the fourth-order Runge-Kutta method. The carrying capacity parameter b is decreased from 1 to -0.2 at time $t = 200$ in this simulation. The red curve shows the tumor size x, the green curve shows $\phi_r := \frac{\partial F}{\partial r}$, and the blue curve shows $\phi_b := \frac{\partial F}{\partial b}$.

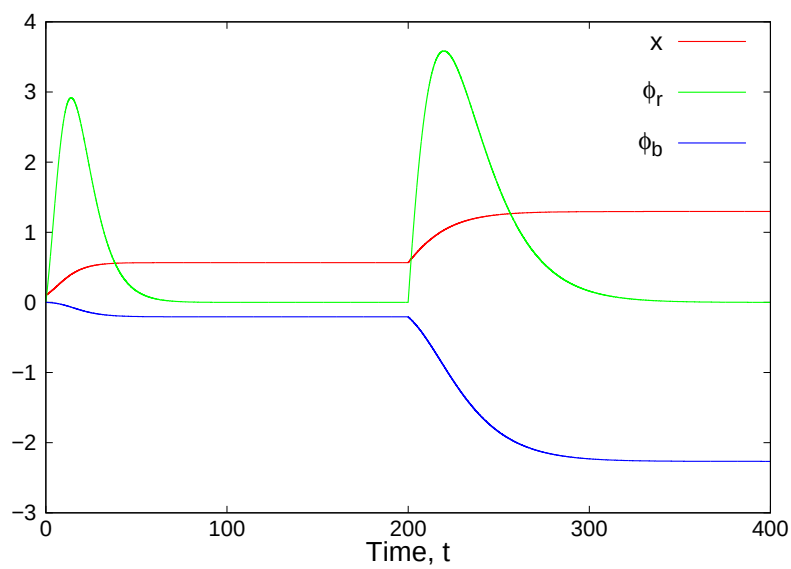

FIGURE 14. Numerical solution to equations 22-24 for $r = 0.082$ and $x_0 = 0.1$. The parameter b is manually changed from 1 to -0.2 at time $t = 200$.

From figure 14, we see that the local sensitivity of the system to changes in r (quantified by ϕ_r) is highest during the period of tumor growth, whereas the ϕ_b is highest in magnitude when the tumor stabilizes at its effective carrying capacity. Furthermore, since the the sign of ϕ_r is positive during tumor growth phases, this shows that increases in r would accelerate

tumor growth at these same times. On the other hand, since the sign of ϕ_b is negative when the tumor size asymptotes, it means that DECREASES in b will accelerate tumor growth upward during these times. It is this interpretation that caused us to shift b downward at time $t = 200$, which stimulates a second growth phase where both $|\phi_r|$ and $|\phi_b|$ are higher than they were during the initial growth phase.

3.3. **Least Squares on Exponential Carrying Capacity Model.** For the exponential carrying capacity model, the parameters r and b can be estimated by the following procedure. Let $\frac{1}{x}\frac{dx}{dt}$ be the new dependent variable, then we have the new equation

$$\frac{1}{x}\frac{dx}{dt} = g(x) = -r\ln\left(xe^{bx}\right) \tag{25}$$

We can approximate $\frac{dx}{dt}$ by a forward difference formula, i.e.

$$\frac{dx}{dt}|_i \approx \frac{x(t_{i+1}) - x(t_i)}{t_{i+1} - t_i}$$

for $i = 0, \ldots, n-1$. At $(n+1)$, the data is not given, so we will assume that $\frac{dx}{dt}|_{i=n} = 0$ by the fact that the curve is flat as t goes to infinity. Now we have different formulation of the least-squares problem, in particular, we want to optimize the following error function

$$\sum_{i=0}^{n-1}\left(g(x(t_i), \vec{\theta}) - \frac{1}{x}\frac{dx}{dt}|_i\right)^2 = 0 \tag{26}$$

where $\vec{\theta} = (r, b)$. Below is the output of fitting the tumor data by using Newton's method for optimization

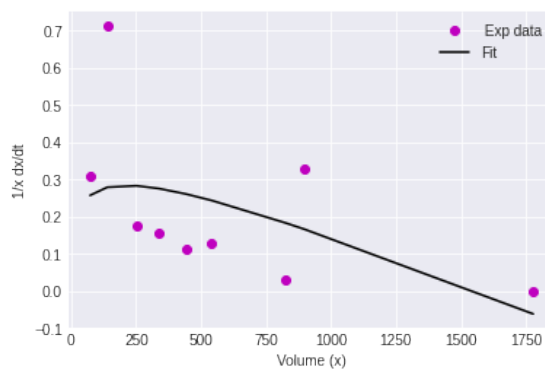

FIGURE 15. Optimized parameters are $r = -0.065246$, $b = -0.004738$ with $RMSE = 873.080485$.

The solution of the original differential equation with these parameters is
We will rescale x by $x^* = log_{10}(x)$ and the DE after rescaling becomes

$$\frac{dx^*}{dt} = \frac{r}{ln(10)} ln(10^{x^*} e^{b10^{x^*}})$$

.

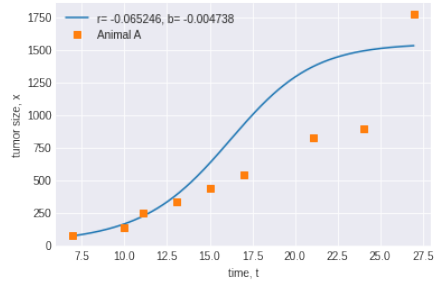

FIGURE 16. Solution of the differential equation (DE) with $r = -0.065246$, $b = -0.004738$.

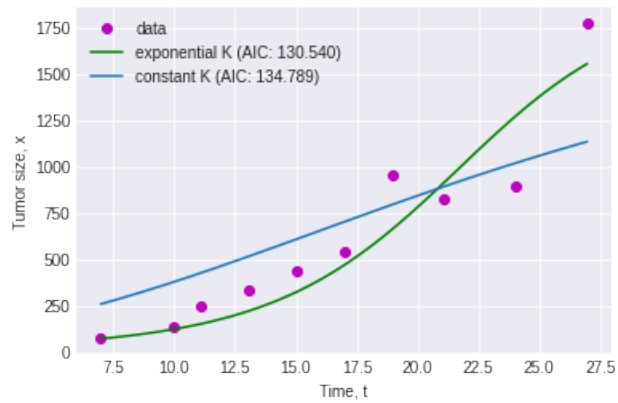

FIGURE 17. Fitting the original data by directly numerically solving the DE. Exponential K: $r = -0.0427$ with 95% CI (-29.1409, 29.0555), $b = -0.00397$ with 95% CI (-4.5172, 4.5093). Constant K: $r = 0.07289$ with 95% CI (-37.8895, 38.0353)

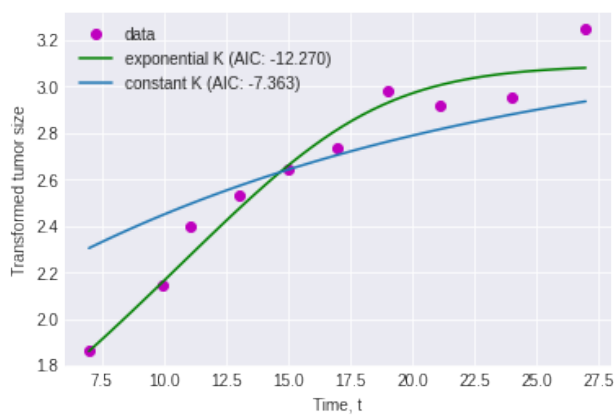

FIGURE 18. Fitting the transformed data. Exponential K: $r = -0.0574$ with 95% CI (-34.8656, 34.7508), $b = -0.00575$ with 95% CI (-5.0196, 5.0081). Constant K: $r = 0.0551$ with 95% CI (-53.3833, 53.4935). **Exponential K model shows smaller AIC value that indicates its better goodness-of-fit.**

.

4. Discussion

The Gompertz model is an effective framework for conceptualizing growth processes, and has proven especially useful for modeling cancerous tumor growth [5, 6]. It differs from the logistic model of Verhulst, in that its phase diagram is asymmetric about the population size at which fastest growth occurs. For a constant growth rate r and carrying capacity K, the logistic model is stated as follows:

$$\dot{x} = rx\left(1 - \frac{x}{K}\right) \tag{27}$$

Whose phase diagram is given in figure 19 below.

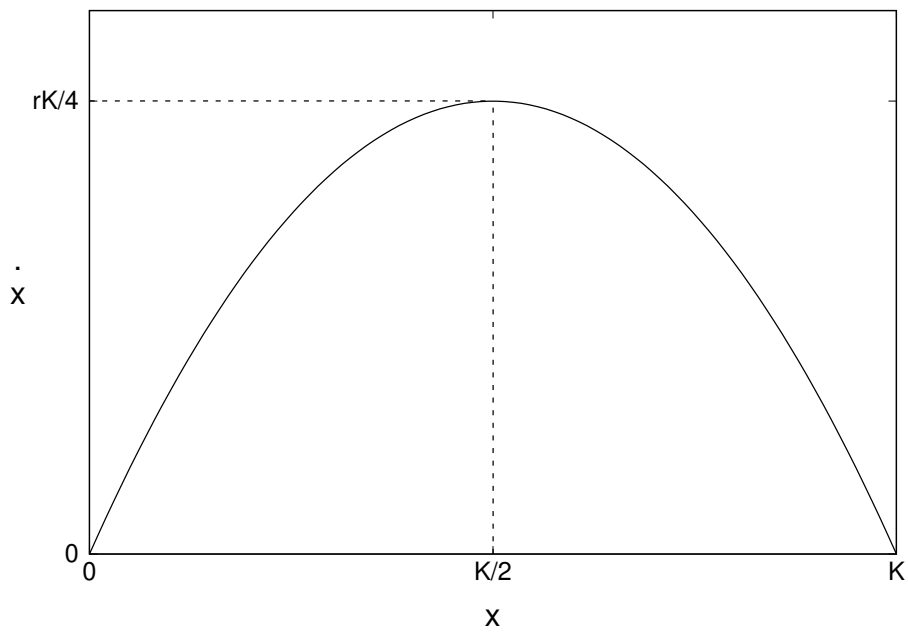

Figure 19. Phase diagram for equation 27

The analytical solution to equation 27 is $x(t) = \frac{K}{1+Ae^{-rt}}$, where $A = \frac{K-x_0}{x_0}$ for initial tumor size x_0. This result contrasts with the analytical solution for the Gompertz equation with constant carrying capacity, initially shown here in equation 2. The Gompertzian growth has $\ddot{x} = 0$ when $x = K/e$, whereas the logistic growth has $\ddot{x} = 0$ when $x = K/2$. In the context of the present biological application, this implies that Gompertzian tumor growth begins to slow down after it has reached tumor size smaller than half of its maximum, whereas logistic growth has an inflection point precisely at the halfway mark.

The modification to the constant K Gompertz model focused on here was the extension to an exponential carrying capacity. This is a simple alteration with respect to the original formulation, and can be fit to tumor time series data by minimizing the sum of squares with respect to the rate of change of tumor size via Newton's method. The carrying capacity quantity b, taken here as a model parameter, controls the relationship between the effective carrying capacity and the current tumor size. Step-like decreases in b were found to induce

repeated tumor growth with long periods of constant size between, which is observed in real cancer data [7]. This work thus forms a foundation for further studies that may take b as a variable, whose corresponding differential equation should yield step-like solutions with some characteristic time between steps. This characteristic time would control the duration between repeated tumor growth, providing a simple mathematical lens through which cancer scientists may understand how long it will be until the next tumor growth occurs in a given organism.

- The exponential carrying capacity model is a simple modification that can produce several types of tumor growth through the parameter b
- A time-dependent b may yield repeated growth or decay, depending on the the sign of Δb.
  - especially relevant to cancer, since tumor growth occurs in phases.

Other future directions of this research may include fitting the same models to other cancer data, to see how well our estimates here generalize to other tumors. If little data are available, a potential substitute or supplement would be to perform stochastic simulations of the models here and see how robust our parameter estimation method is. It would be informative to see how these results fare in the face of various noise structures. In particular, if there is any evidence of heterogeneous variance in the data (whether *in vivo* or *in silico*), the method of weighted least-squares is a potential substitute for the OLS method used here.

Outside of these mathematical details above, it is imperative for further studies to investigate the biophysical origin of the carrying capacity parameter b. What tumor characteristics predict the value of b? Is b related to a concentration of some chemical that promotes tumor growth? It is likely an aggregate of many factors, including the geometry of the environment in which the tumor grows. Besides volume, the surface area of the tumor may also be a quantity that is relevant to its potential to grow or shrink.

The future directions of this research can be the followings

- Fit model to other data sets.
- Simulation experiments of Gompertz model, perform sensitivity analysis for different noise types.
- Write differential equation for b with periodic solution.
- Investigate possible biological/physical meaning of parameter b in the exponential K model.

References

[1] U.S. Cancer Statistics Working Group. U.S. Cancer Statistics Data Visualizations Tool, based on 2019 submission data (1999-2017): U.S. Department of Health and Human Services, Centers for Disease Control and Prevention and National Cancer Institute, www.cdc.gov/cancer/dataviz, accessed: 2021-03-14.

[2] C. Vaghi, A. Rodallec, R. Fanciullino, J. Ciccolini, J. P. Mochel, M. Mastri, C. Poignard, J. M. Ebos, S. Benzekry, Population modeling of tumor growth curves and the reduced Gompertz model improve prediction of the age of experimental tumors, PLoS computational biology 16 (2) (2020) e1007178.

[3] R. A. Beckman, I. Kareva, F. R. Adler, How should cancer models be constructed?, Cancer Control 27 (1) (2020) 1073274820962008.

[4] A. Capaldi, S. Behrend, B. Berman, J. Smith, J. Wright, A. L. Lloyd, Parameter estimation and uncertainty quantication for an epidemic model, Mathematical biosciences and engineering (2012) 553.

[5] J. Aroesty, T. Lincoln, N. Shapiro, G. Boccia, Tumor growth and chemotherapy: Mathematical methods, computer simulations, and experimental foundations, Mathematical Biosciences 17 (3-4) (1973) 243–300.

[6] R. Mehdizadeh, S. P. Shariatpanahi, B. Goliaei, S. Peyvandi, C. Rüegg, Dormant tumor cell vaccination: A mathematical model of immunological dormancy in triple-negative breast cancer, Cancers 13 (2) (2021) 245.

[7] H. Enderling, N. Almog, L. Hlatky, Systems biology of tumor dormancy, Vol. 734, Springer Science & Business Media, 2012.

5. Appendix

5.1. C++ code that produces sensitivity system data in figure 14.

```
#include<iostream>
#include<stdio.h>
#include<vector>
#include<math.h>
#include<cmath>
#include<algorithm>
#include<unistd.h>
#include<sys/stat.h>
#include<sys/types.h>
#include<string.h>

using namespace std;

//--- Runge-Kutta numerical method settings ---
#define N     3       /* Number of first order differential equations */
#define h1    0.01     /* Step of integration in seconds */
#define MAX   500       /* Max for running time */
#define trans 0     /* Transient time */

//--- Gompertz model parameters ---//
#define r    .082        // growth rate
#define x0   0.1        // initial tumor size

FILE *fp1 ; /* Internal filenames */

void runge4(double x, double y[], double step);   /* Runge-Kutta function */

double f(double x, double y[], int i);  // Setting function for Gompertz equation

double b(double z); // carrying capacity parameter (function of time here)

int main()
{
    double t, y[N] ;   // Real variables
    int j;      // Integer

    fp1=fopen("data.txt", "w");  /* Creates file for data recording */

    y[0]=  x0;      /* Defining initial conditions */
    y[1]=   0;
    y[2]=   0;

    /* Time loop */

    for (j=0; j*h1<=MAX; j++)
    {
        t=j*h1; //discretizes time
        runge4(t, y, h1); // Calls RungeKutta function

        if (t>trans) // not recording transient data
        {
                        //Print time, xi, and yi at every iteration
            fprintf(fp1, "%6.2lf\t %9.3lf\t %9.3lf\t %9.3lf\n", t-trans, y[0], y[1], y[2]);
        }
    }
```

```

    fclose(fp1);  /* Close data file */

} // end of main() loop

// Solving the equations numerically
void runge4(double x, double y[], double step)
{
    double h = step/2.0,
    t1[N], t2[N], t3[N],         //Temporary storage arrays for RK4 algorithm
    k1[N], k2[N], k3[N],k4[N];
    int i;

    for (i=0;i<N;i++) t1[i]=y[i]+0.5*(k1[i]=step*f(x, y, i));
    for (i=0;i<N;i++) t2[i]=y[i]+0.5*(k2[i]=step*f(x+h, t1, i));
    for (i=0;i<N;i++) t3[i]=y[i]+    (k3[i]=step*f(x+h, t2, i));
    for (i=0;i<N;i++) k4[i]=                step*f(x+step, t3, i);

    for (i=0;i<N;i++) y[i]+=(k1[i]+2*k2[i]+2*k3[i]+k4[i])/6.0;
}

// Gompertz model w/ nonlinear carrying capacity
double  f(double x, double y[], int i)
{
    // tumor size
    if (i==0) return -r*y[0]*(log(y[0]) + b(x)*y[0]);
    // phi_r = sensitivity of tumor size derivative to growth rate r
    if (i==1) return -r*(log(y[0])+1+2*b(x)*y[0])*y[1]-y[0]*log(y[0])-b(x)*pow(y[0],2);
    // phi_b = sensitivity of tumor size derivative to carrying capacity parameter b
    if (i==2) return -r*(log(y[0])+1+2*b(x)*y[0])*y[2]-r*pow(y[0],2);
    return 0;
}

double b(double z) // carrying capacity parameter b
{
    if (z - trans > 200) return -0.2;
    else return 1.;
}
```

5.2. C code that produces bifurcation diagram in figure 6.

```
#include <stdio.h>
#include <math.h>

/* Parameters */

double   r  = 0.082 ; // growth rate
double   x0 = 0.020 ; // initial tumor size
double  K0 = 2171. ; // carrying capacity when tumor size = 0
double   b          ; // bifurcation parameter

/* Variables */
int     i ;
double  x ;
FILE    *fp ;

/* Program ------------------------------------------------------ */

// Gompertz equation w/ nonlinear carrying capacity
double f1(double t,double x)
{
double ret;
ret = -r * x * log(x/(K0*exp(-b*x)));
return ret;
}

/* RUNGE-KUTTA 4TH ORDER ROUTINE */

void RK4(double x)
{
double    h,t,
          k11,k21,k31,k41,k12,k22,k32,k42,
          k13,k23,k33,k43,oldx,oldy,oldz ;
int       i;

h = 0.005;
t = 0.0;
i = 0;

    for(b=0.01; b<=5.0; b+=0.01)
    {
        i=0;
        while (i < 50000)
        {
        k11 = h * f1(t,x);
        k21 = h * f1(t+h/2, x+k11/2);
        k31 = h * f1(t+h/2, x+k21/2);
        k41 = h * f1(t+h, x+k31);

        oldx = x;

        x = x + (k11 + 2*k21 + 2*k31 + k41) / 6;
        t = t + h;
        i++ ;

   if (i>40000 && x>0)
     fprintf(fp,"%6.3lf %6.3lf\n", b,x);
        }//end while
```

```
    }//end for
}
int main()
{
   fp = fopen("Gom-b_bif.txt","w");
   x = x0;
   RK4(x);
   fclose(fp);
}
```

5.3. R code for OLS fit and Breusch-Pagan test for heteroscedasticity of constant K model in figure 12.

```
 # import data
mydata <- read.table("D:/C-drive/ISU_Grad/MAT-442/project/codes/OLS_GomExpConstK/ABCmod.txt",
                     header=FALSE, col.names = c("time","size"))


######################### Ordinary Least Squares ###########################

# construct and display model
ols<-lm(size~time,data=mydata)
summary(ols)
confint(ols,level=0.95) #prints confidence intervals for slope and intercept
fitted(ols)[0]

# Test for heteroscedasticity

# create residual versus fitted plot
plot(fitted(ols),resid(ols),xlab='Fitted Values',ylab='Residuals')
abline(0,0)

# To formally test for heteroscedasticity, use Breusch-Pagan test

# load lmtest package
library(lmtest)

# perform Breusch-Pagan test
bptest(ols) # no significance -> no heteroscedasticity
```

5.4. Python code: fitting the data with the exponential carrying capacity model.

```python
import numpy as np
import matplotlib.pyplot as plt
import scipy as sc
from scipy.integrate import odeint
from scipy.optimize import fsolve
from shapely.geometry import LineString
plt.style.use('seaborn-darkgrid')

from scipy.optimize import curve_fit

import pandas as pd
from numpy.linalg import inv

# %matplotlib inline

from scipy import linalg
from scipy.optimize import minimize
import sympy as sp
from sympy import Sum, var, solve, nsolve
from sympy.core.symbol import symbols
from sympy.solvers.solveset import nonlinsolve
from sympy import Matrix
from sympy.matrices import Matrix, eye, zeros, ones, diag, GramSchmidt

"""Case 1: g(x) = bx+c"""

#Equilbrium point(s)

b = 0.2
c = 0.1

lhs = lambda x: np.log(x)
rhs = lambda x, b, c: np.log(b*x+c)
t = np.linspace(0.01,20, 1000)

point = c/(1-b)

plt.plot(t,lhs(t), label ='ln(x)')
plt.plot(t,rhs(t,b,c), label ='ln(bx+c)')
plt.plot(point, lhs(point), 'o', label = 'equilibrium point %.3f' %(point))
plt.xlabel('x')
plt.ylabel('function')
plt.legend()
plt.show()

#Solution of Gompertz model

a_0 = 0.5
a_1 = 1
a_2 = 1.5

def f1(x,t,a1):
    dxdt = -a1*x*np.log(x/(b*x+c))
    return dxdt

x_0 = 0.1

x0 = odeint(f1,x_0,t,args=(a_0,))
x1 = odeint(f1,x_0,t,args=(a_1,))
x2 = odeint(f1,x_0,t,args=(a_2,))

# plot results
```

```
plt.axhline(y=point, color='k', linestyle='--', label = 'equilibrium')
plt.plot(t,x0, label = 'a=0.5, b=0.2, c=0.1')
plt.plot(t,x1, label = 'a=1, b=0.2, c=0.1')
plt.plot(t,x2, label = 'a=1.5, b=0.2, c=0.1')
plt.xlabel('time, t')
plt.ylabel('tumor size, x')
plt.legend()
plt.show()

"""Case 2: g(x) = bx^2+cx+d"""

#Equilbrium point(s)

b = -0.5
d = 0
c = 2*np.sqrt(b*d)+3

lhs1 = lambda x: np.log(x)
rhs1 = lambda x, b, c, d: np.log(b*x**2+c*x+d)
t = np.linspace(0.01,20, 10000)

point1 = (-(c-1)+np.sqrt((c-1)**2-4*b*d))/(2*b)
point2 = (-(c-1)-np.sqrt((c-1)**2-4*b*d))/(2*b)

plt.plot(t,lhs1(t), label ='ln(x)')
plt.plot(t,rhs1(t,b,c,d), label ='ln(bx^2+cx+d)')
plt.plot(point1, lhs1(point1), 'o', label = 'equilibrium point %.d' %(point1))
plt.plot(point2, lhs1(point2), 'o', label = 'equilibrium point %.3f' %(point2))
plt.xlabel('x')
plt.ylabel('function')
plt.legend()
plt.show()

#Solution of Gompertz model

a_0 = 0.5
a_1 = 1
a_2 = 1.5

def f1(x,t,a1):
    dxdt = -a1*x*np.log(x/(b*x**2+c*x+d))
    return dxdt

x_0 = 0.01

x0 = odeint(f1,x_0,t,args=(a_0,))
x1 = odeint(f1,x_0,t,args=(a_1,))
x2 = odeint(f1,x_0,t,args=(a_2,))

# plot results
plt.axhline(y=point2, color='k', linestyle='--', label = 'equilibrium point 1')
plt.axhline(y=point1, color='r', linestyle='--', label = 'equilibrium point 2')
plt.plot(t,x0, label = 'a= %.1f, b= %.1f, c= %.1f, d = %.1f' %(a_0, b, c, d))
plt.plot(t,x1, label = 'a= %.1f, b= %.1f, c= %.1f, d = %.1f' %(a_1, b, c, d))
plt.plot(t,x2, label = 'a= %.1f, b= %.1f, c= %.1f, d = %.1f' %(a_2, b, c, d))
plt.xlabel('time, t')
plt.ylabel('tumor size, x')
plt.legend(loc='best')
plt.show()

"""Case 3: g(x) = bln(cx)"""

b = -2
c = 0.5
```

```python


lhs2 = lambda x: x
rhs2 = lambda x, b, c: np.log((c*x)**b)
t = np.linspace(0.1,14, 10000)


f = lambda x: np.log((c*x)**b) -x

i_guess = 0.1
point1 = fsolve(f, i_guess)

plt.plot(t,lhs2(t), label ='x')
plt.plot(t,rhs2(t,b,c), label ='bln(cx)')
plt.plot(point1[0], lhs2(point1[0]), 'o', label = 'equilibrium point %.3f' %(point1[0]))

plt.xlabel('x')
plt.ylabel('function')
plt.legend()
plt.show()

#Solution of Gompertz model

a_0 = 0.5
a_1 = 1
a_2 = 1.5

def f1(x,t,a1):
    dxdt = -a1*x*np.log(x/(np.log((c*x)**b)))
    return dxdt

x_0 = 0.01

x0 = odeint(f1,x_0,t,args=(a_0,))
x1 = odeint(f1,x_0,t,args=(a_1,))
x2 = odeint(f1,x_0,t,args=(a_2,))

# plot results
plt.axhline(y=point1[0], color='k', linestyle='--', label = 'equilibrium')
plt.plot(t,x0, label = 'a= %.1f, b= %.1f, c= %.1f' %(a_0, b, c))
plt.plot(t,x1, label = 'a= %.1f, b= %.1f, c= %.1f' %(a_1, b, c))
plt.plot(t,x2, label = 'a= %.1f, b= %.1f, c= %.1f' %(a_2, b, c))
plt.xlabel('time, t')
plt.ylabel('tumor size, x')
plt.legend()
plt.show()

"""Case 4: g(x) = e^-(bx+c)"""

b = 1
c = 0

lhs = lambda x, b, c: b*x + c
rhs = lambda x: -np.log(x)
t = np.linspace(0.01,20, 1000)

plt.plot(t,lhs(t, b, c), label ='b*x + c')
plt.plot(t,rhs(t), label ='-np.log(x)')
#plt.plot(point, lhs(point), 'o', label = 'equilibrium point %.3f' %(point))
plt.xlabel('x')
plt.ylabel('function')
plt.legend()
plt.show()

```

```
b1 = -0.004738
b2 = -0.35
b3 = 0.05
c = 0

lhs = lambda x, b, c: b*x + c
rhs = lambda x: -np.log(x)
t = np.linspace(0.01,5, 1000)

plt.plot(t,lhs(t, b1, c), label ='b = %.3f' %(b1))
plt.plot(t,lhs(t, b2, c), label ='b = %.3f' %(b2))
plt.plot(t,lhs(t, b3, c), label ='b = %.3f' %(b3))
plt.plot(t,rhs(t), label ='-np.log(x)')
#plt.plot(point, lhs(point), 'o', label = 'equilibrium point %.3f' %(point))
plt.xlabel('x')
plt.ylabel('function')
plt.legend()
plt.show()

dfA = pd.read_csv('/content/drive/MyDrive/Projects/BioMath research project/A.csv', header=None)
dfB = pd.read_csv('/content/drive/MyDrive/Projects/BioMath research project/B.csv', header=None)
dfC = pd.read_csv('/content/drive/MyDrive/Projects/BioMath research project/C.csv', header=None)

dfA.columns = ['Time', 'Volume']
dfB.columns = ['Time', 'Volume']
dfC.columns = ['Time', 'Volume']

display(dfA, dfB, dfC)

x1 = dfA['Time']
y1 = dfA['Volume']
x2 = dfB['Time']
y2 = dfB['Volume']
x3 = dfC['Time']
y3 = dfC['Volume']

x1 = x1.drop([6])
y1 = y1.drop([6])

x1 = np.array(x1)
y1 = np.array(y1)

plt.plot(x1,y1, 'o', label = 'Animal A')

plt.legend()
plt.xlabel('Time (days)')
plt.ylabel('Volume (mm^3)')
plt.show()

der1 = []
for i in range(len(x1)-1):
  der1.append((y1[i+1]-y1[i])/((x1[i+1]-x1[i])*y1[i]))
der1.append(0)

plt.plot(y1,der1,'go')
plt.show()

def CI(est, fit, x, n, m):
  return est - 2.306*np.sqrt(sum([(i-j)**2/n for i in x for j in fit])*m), est + 2.306*np.sqrt(sum([(i-j)**2/n for i in x for

def lower(fit, x, n):
  return fit - 1.86*se(fit, x, n)

```

```
def upper(fit, x, n):
  return fit + 1.86*se(fit, x, n)

"""#Newton's method"""

def N(xdata, ydata, iv, tol, max):

  m = len(xdata)

  def fc(x, p):
    return np.log(1/(x**p[0]*np.exp(p[0]*p[1]*x)))

  def rms(y, yfit):
    return np.sqrt(np.sum((y-yfit)**2)/(m-2))



  a,b = symbols('a b', real = True)

  eq1 = Matrix([sp.Add(*[(sp.log(1/(xdata[i]**a*sp.exp(a*b*xdata[i])))-ydata[i])**2 for i in range(m)])])
  params = Matrix([a, b])

  grad = eq1.jacobian(params)

  vector_f = Matrix([grad[0], grad[1]])
  H = vector_f.jacobian(params)

  it = 0
  itlist = [0]
  grad_norm = []

  def fceval(ivn):
    return sum([(np.log(1/(x**ivn[0]*np.exp(ivn[0]*ivn[1]*x)))-h)**2 for x in xdata for h in ydata])


  diff = fceval(iv)
  for i in range(max):
    f = np.array(vector_f.subs({a:iv[0], b:iv[1]})).astype('float').ravel()
    grad_norm.append(np.linalg.norm(f))
    print(diff)
    if diff < tol:
      aprlist = [np.log(1/(x**iv[0]*np.exp(iv[0]*(iv[1]*x)))) for x in xdata]
      print('Convergence was reached at tolerance = %f and %d iterations' %(tol, it))
      print('Solution = {a = %f, b = %f}' %(iv[0], iv[1]))
      print('RMSE is %f' %(rms(y1, fc(xdata,iv))))
      plt.fill_between(xdata, lower(aprlist, ydata,len(xdata)-len(iv)), upper(aprlist, ydata,len(xdata)-len(iv)), color='b', a
      plt.plot(xdata,ydata, 'mo', label = ' Exp data')
      plt.plot(xdata,aprlist, color = 'k', label = 'Fit')
      plt.xlabel('Volume (x)')
      plt.ylabel('1/x dx/dt')
      plt.legend()
      plt.draw()
      print(grad_norm)
      plt.figure()
      plt.plot(itlist, grad_norm, 'g')
      plt.axhline(y=tol, color='y', linestyle='--', label = 'Tolerance level')
      plt.legend()
      plt.xlabel('Iterations')
      plt.ylabel('2-norm of grad')
      plt.draw()
      break

    else:
      it += 1
```

```
        itlist.append(int(it))
        A = H.subs({a:iv[0], b:iv[1]})
        A = np.array(A).astype('float')
        h = inv(A).dot(-f)
        ivnew = iv + h

        diff = abs(fceval(ivnew) - fceval(iv))

        iv = ivnew

        if i == int(max-1):
          aprlist = [np.log(1/(x**iv[0]*np.exp(iv[0]*(iv[1]*x)))) for x in xdata]
          print('Maximum number of %d iterations is reached' %(it))
          print('Parameters = {a = %f, b = %f}' %(iv[0], iv[1]))
          plt.plot(xdata,ydata, 'mo', label = ' Exp data')
          plt.plot(xdata,aprlist, color = 'k', label = 'Approximation')
          plt.xlabel('Volume (x)')
          plt.ylabel('1/x dx/dt')
          plt.legend()
          plt.draw()
          print(grad_norm)
          plt.figure()
          plt.plot(itlist[:-1], grad_norm, color = 'g')
          plt.axhline(y=tol, color='y', linestyle='--', label = 'Tolerance level')
          plt.legend()
          plt.xlabel('Iterations')
          plt.ylabel('2-norm of grad')
          plt.draw()
          break
    a = iv[0]
    b = iv[1]

    def f1(x,t,a1,b):
        dxdt = -a1*x*np.log(x*np.exp(b*x))
        return dxdt

    x_0 = y1[0]
    t = x1
    x0 = odeint(f1,x_0,t,args=(a, b))

    # plot results
    #plt.axhline(y=, color='k', linestyle='--', label = 'equilibrium')
    plt.figure()
    #plt.fill_between(xdata, lower(x0, ydata,len(xdata)-len(iv)), upper(x0, ydata,len(xdata)-len(iv)), color='b', alpha=.1)
    plt.plot(t,x0, label = 'r= %f, b= %f' %(a, b))
    plt.plot(x1, y1, 's', label = 'Animal A')
    plt.xlabel('time, t')
    plt.ylabel('tumor size, x')
    plt.legend(loc = 'best')
    plt.draw()
    plt.show()

ig = np.array([0.2, -0.001])
N(xdata = y1, ydata = der1, iv = ig, tol = 1E-15, max = 100)

n = len(x1)
t = x1

def rms(y, yfit):
    return np.sqrt(np.sum((y-yfit)**2)/(n-2))

def fitfunc(t, r, b):
  def myode(y, t):
```

```
    return -r*y*np.log(y*np.exp(b*y))

  y0 = y1[0]
  ysol = odeint(myode, y0, t)
  return ysol[:,0]

p0 = -0.05, -0.007
fit1, cov = curve_fit(fitfunc, t, y1, p0)
yfit = fitfunc(x1, *fit1)
print(fit1)
a1,b1 = CI(fit1[0], y1, yfit, len(x1)-2, cov[0][0])
a2,b2 = CI(fit1[1], y1, yfit, len(x1)-2, cov[1][1])
print('CI for r is (%f, %f)' %(a1,b1))
print('CI for b is (%f, %f)' %(a2,b2))
tspace = np.linspace(t[0], t[len(x1)-1])
fit = fitfunc(tspace, *fit1)
fit1 = fitfunc(t,*fit1)
xi = 0.5

sigma = [1/(x**(2*xi)) for x in fit1]

fitw, covw = curve_fit(fitfunc, t, y1, p0, sigma=sigma, absolute_sigma=True)
yfitw = fitfunc(x1, *fitw)
print(fitw)
a1,b1 = CI(fitw[0], y1, yfitw, len(x1)-2, covw[0][0])
a2,b2 = CI(fitw[1], y1, yfitw, len(x1)-2, covw[1][1])
print('CI for r is (%.10f, %.10f)' %(a1,b1))
print('CI for b is (%.10f, %.10f)' %(a2,b2))
fitw = fitfunc(tspace, *fitw)

plt.plot(t, y1, 'mo', label='data')
plt.plot(tspace, fit, 'b-', label='unweighted fit')
plt.plot(tspace, fitw, 'g-', label='weighted fit')
plt.ylabel('Tumor size, x')
plt.xlabel('Time, t')
plt.legend(loc='best')
plt.show()

n = len(x1)
t = x1
y1log = np.log10(y1)

def rms(y, yfit):
    return np.sqrt(np.sum((y-yfit)**2)/(n-2))

def fitfunc(t, r, b):
  def myode(y, t):
    return -r*np.log(10**y*np.exp(b*10**y))/np.log(10)

  y0 = y1log[0]
  ysol = odeint(myode, y0, t)
  return ysol[:,0]

p0 = -0.05, -0.007
fit1, cov = curve_fit(fitfunc, t, y1log, p0)
yfit = fitfunc(x1, *fit1)
print(fit1)
a1,b1 = CI(fit1[0], y1, yfit, len(x1)-2, cov[0][0])
a2,b2 = CI(fit1[1], y1, yfit, len(x1)-2, cov[1][1])
print('CI for r is (%f, %f)' %(a1,b1))
print('CI for b is (%f, %f)' %(a2,b2))
tspace = np.linspace(t[0], t[len(x1)-1])
fit = fitfunc(tspace, *fit1)
```

```
fit1 = fitfunc(t,*fit1)

xi = 4

sigma = [1/(x**(2*xi)) for x in fit1]

fitw, covw = curve_fit(fitfunc, t, y1log, p0, sigma=sigma, absolute_sigma=True)
yfitw = fitfunc(x1, *fitw)
print(fitw)
a1,b1 = CI(fitw[0], y1, yfitw, len(x1)-2, covw[0][0])
a2,b2 = CI(fitw[1], y1, yfitw, len(x1)-2, covw[1][1])
print('CI for r is (%f, %f)' %(a1,b1))
print('CI for b is (%f, %f)' %(a2,b2))
fitw = fitfunc(tspace, *fitw)

plt.plot(t, y1log, 'mo', label='data')
plt.plot(tspace, fit, 'b-', label='unweighted fit')
plt.plot(tspace, fitw, 'g-', label='weighted fit')
plt.ylabel('Transformed tumor size')
plt.xlabel('Time, t')
plt.legend(loc='best')
plt.show()
```

Department of Mathematics, Illinois State University, Normal, IL USA 61790-4520